

Learn Three.js basics

u r my code tutor can u teach three.js i'm beginner

Perfect 🙌 — I'd *love* to teach you Three.js.

Let's treat this like a **step-by-step course** where you actually *build cool 3D stuff* while learning.

Goal

You'll go from **beginner** → **confident 3D web dev**, able to make:

- 3D landing pages (like Apple or Tesla websites)
 - Interactive scenes (with models, lights, animation, camera controls)
 - Projects that combine **Three.js + GSAP + React/Next.js**
-

Step 0 – Before We Start

To confirm — tell me:

1. Do you know **JavaScript (ES6)** basics already? (like functions, import/export, classes)
2. Do you want to use **pure Three.js** first or **with React (via react-three-fiber)** later?
3. Are you coding in **VS Code + local browser**, or **online (like CodeSandbox)**?

Once I know this, I'll set up your first project and teach you:

- What is WebGL & how Three.js makes it easy
 - How to set up a 3D scene (Scene, Camera, Renderer)
 - Add objects, lights, and animation loops
-

When you reply, I'll give you **Lesson 1: "Your First 3D Scene"** 🧱

So — answer the 3 setup questions first 🙌

pure Three.js i know js vs code

Perfect 🔥 That's exactly what I needed.

You're ready for **Lesson 1: Your First 3D Scene** — pure Three.js style.

Lesson 1 – Create Your First 3D Scene

We'll make:

A simple 3D cube rotating in space 

Step 1: Project Setup

1. **Make a folder** anywhere — e.g. `threejs-basics`
2. Inside it, create these files:

CSS

`index.html`

`main.js`

3. Open the folder in **VS Code**
4. Install a local server so you can run it properly (Three.js won't run via `file://`):

bash

`npm install -g live-server`

Then run:

bash

`live-server`

It'll open your project in the browser.

Step 2: Add HTML

In `index.html`:

html

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>Three.js Beginner</title>
  </head>
  <body style="margin:0; overflow:hidden;">
    <script type="module" src="main.js"></script>
  </body>
</html>
```

Step 3: Add JavaScript

In `main.js`:

```
js

// Import Three.js from CDN (no install needed yet)
import * as THREE from 'https://cdn.jsdelivr.net/npm/three@0.161.0/build/three.module.js';

// 1 Scene
const scene = new THREE.Scene();

// 2 Camera
const camera = new THREE.PerspectiveCamera(
  75, window.innerWidth / window.innerHeight, 0.1, 1000
);
camera.position.z = 3;

// 3 Renderer
const renderer = new THREE.WebGLRenderer({ antialias: true });
renderer.setSize(window.innerWidth, window.innerHeight);
document.body.appendChild(renderer.domElement);

// 4 Create a Cube
const geometry = new THREE.BoxGeometry(1, 1, 1);
const material = new THREE.MeshStandardMaterial({ color: 0x00ff00 });
const cube = new THREE.Mesh(geometry, material);
scene.add(cube);
```

```
// 5 Add a Light
const light = new THREE.PointLight(0xffffff, 1);
light.position.set(5, 5, 5);
scene.add(light);

// 6 Animation Loop
function animate() {
  requestAnimationFrame(animate);
  cube.rotation.x += 0.01;
  cube.rotation.y += 0.01;
  renderer.render(scene, camera);
}
animate();

// 7 Resize handling
window.addEventListener('resize', () => {
  camera.aspect = window.innerWidth / window.innerHeight;
  camera.updateProjectionMatrix();
  renderer.setSize(window.innerWidth, window.innerHeight);
});
```

Now run `live-server` — you'll see a **rotating green cube** 

Congrats — you just rendered your **first 3D object**!

What You Just Did

- `Scene` → 3D world container
- `Camera` → how you view the world
- `Renderer` → draws everything to your screen
- `Mesh` = `Geometry` (shape) + `Material` (appearance)
- `Light` → makes materials visible
- `animate()` → creates a continuous render loop

Challenge for You

Try to:

1. Change cube color
2. Change camera distance

3. Add another cube beside it

When you've done that, tell me 

Then we'll go to **Lesson 2: Camera controls, Orbit, and Lighting mastery** ✨

yess

🔥 Awesome! You've officially rendered your first 3D object — that's a big milestone. Now you're ready for **Lesson 2: Camera Controls + Lighting Mastery** 🎮💡

Lesson 2 – Make the Scene Interactive + Beautiful

Goal:

You'll add **mouse control** to orbit the camera and explore the scene, and understand how different **lights** change the mood.

Step 1: Add Orbit Controls

OrbitControls let you move the camera by dragging with the mouse.

In your `main.js`, add this import (below the Three.js import):

```
js

import { OrbitControls } from
'https://cdn.jsdelivr.net/npm/three@0.161.0/examples/jsm/controls/OrbitControls.js';
```

Then, after creating the camera and renderer, add:

```
js

// Controls
const controls = new OrbitControls(camera, renderer.domElement);
controls.enableDamping = true; // smooth motion
```

Now, in your `animate()` loop, update it:

```
js
```

```
function animate() {
  requestAnimationFrame(animate);
  cube.rotation.x += 0.01;
  cube.rotation.y += 0.01;
  controls.update(); // required for damping
  renderer.render(scene, camera);
}
```

✓ Try it: You can now **click + drag** to orbit around the cube and **scroll** to zoom in/out.

🧩 Step 2: Play with Lights

Let's explore different lighting types.

Replace your old `PointLight` code with this:

```
js

// 💡 Directional Light - like the sun
const dirLight = new THREE.DirectionalLight(0xffffff, 1);
dirLight.position.set(2, 2, 5);
scene.add(dirLight);

// 💡 Ambient Light - soft global light
const ambientLight = new THREE.AmbientLight(0xffffff, 0.3);
scene.add(ambientLight);
```

You can tweak:

```
js

dirLight.intensity = 1.5;
ambientLight.intensity = 0.2;
```

👉 `DirectionalLight` = casts parallel rays (like sunlight)

👉 `AmbientLight` = fills in shadows, softens dark areas

🧩 Step 3: Change Material for Realism

Let's make the cube metallic:

```
js

const material = new THREE.MeshStandardMaterial({
  color: 0x0077ff,
  metalness: 0.7,
  roughness: 0.3,
});
```

Try adjusting:

- `metalness` between `0 → 1`
- `roughness` between `0 → 1`

You'll see how the light interacts with it!

Concept Recap

Concept	Meaning
OrbitControls	lets user move camera with mouse
DirectionalLight	sunlight-style shadows
AmbientLight	soft fill light
Metalness/Roughness	how shiny or dull material looks

Mini Challenge

Do this before Lesson 3:

1. Add **two cubes** — each with different material colors/metalness
 2. Add a **rotation speed control** (e.g., faster or slower spin)
 3. Try adding a **different light color** (like soft blue or orange)
-

Once you've done that, reply with **"done"** ✅,
and we'll start **Lesson 3: Textures + Backgrounds + Realistic Materials** ✨

done

🔥 Excellent, you're progressing *fast*!

By now you've got:

- ✅ a working 3D scene
- ✅ interactive camera controls
- ✅ lighting that makes it feel alive

Time to level up again —

Welcome to **Lesson 3: Textures, Backgrounds & Realistic Materials** 🎨✨

🎯 Goal

You'll learn to:

- Apply **image textures** to objects
- Add **backgrounds (sky or space)**
- Make surfaces look **realistic** (wood, metal, etc.)

🧩 Step 1: Add a Texture

Textures = images wrapped around geometry.

At the top of your `main.js`, add:

js

```
const textureLoader = new THREE.TextureLoader();  
const texture = textureLoader.load('https://threejs.org/examples/textures/brick_diffuse.jpg');
```

Then use this texture in your material:

js

```
const material = new THREE.MeshStandardMaterial({  
  map: texture,
```



```
metalness: 0.2,  
roughness: 0.8,  
});
```

💡 Tip: Try different textures:

- https://threejs.org/examples/textures/uv_grid_opengl.jpg
- https://threejs.org/examples/textures/brick_diffuse.jpg
- https://threejs.org/examples/textures/earth_atmos_2048.jpg

You'll see your cube instantly look like **brick**, **metal**, or **Earth** 🌍.

🧩 Step 2: Add a Background (Environment)

Option 1 — Simple Color Background

```
js  
  
scene.background = new THREE.Color(0x222233);
```

Option 2 — Image Background

```
js  
  
const bgTexture =  
textureLoader.load('https://threejs.org/examples/textures/skybox/sky.jpg');  
scene.background = bgTexture;
```

Option 3 — Cube Texture (Skybox) 🌌

This makes a real 3D environment around your scene:

```
js  
  
const loader = new THREE.CubeTextureLoader();  
const urls = [  
  'https://threejs.org/examples/textures/cube/skybox/px.jpg',  
  'https://threejs.org/examples/textures/cube/skybox/nx.jpg',  
  'https://threejs.org/examples/textures/cube/skybox/py.jpg',  
  'https://threejs.org/examples/textures/cube/skybox/ny.jpg',  
  'https://threejs.org/examples/textures/cube/skybox/pz.jpg',  
  'https://threejs.org/examples/textures/cube/skybox/nz.jpg',  
];
```

```
const cubeTexture = loader.load(urls);
scene.background = cubeTexture;
```

Now you have a **sky environment** — move the camera around with OrbitControls and enjoy 🌞

🧩 Step 3: Combine Everything

Try this:

Replace your cube with a **Sphere** (looks better with textures):

```
js

const geometry = new THREE.SphereGeometry(1, 32, 32);
```

Now you've got something like a **planet or shiny orb** floating in 3D space.

🧠 Key Concepts

Concept	Description
Texture	2D image applied to 3D surface
CubeTexture	6-sided environment image
TextureLoader	Loads image textures
Roughness/Metalness	Define how light interacts with surface
Background	Defines what's behind your 3D objects

✅ Mini Challenge

Before Lesson 4, try:

1. Use a **different texture** (your own JPG/PNG — put in project folder)

2. Change the background color or image
 3. Try **sphere + texture** to make a mini **Earth** 🌍
-

Once you've done that, reply **"done"** ✅

and we'll go to **Lesson 4: Animation + Movement + GSAP Integration** — where things start *moving dynamically*! ⚡

done

Let's gooo 🚀 — you've now covered the 3D **visual foundations**:

- ✅ Geometry
- ✅ Materials & Lighting
- ✅ Textures & Backgrounds
- ✅ Camera control

That's 80% of what powers all 3D websites you see!

Now comes the fun part: **Lesson 4 – Animation, Movement & GSAP Integration** 🎬



🎯 Goal

You'll learn to:

- Move and animate objects smoothly
 - Control animations (start, stop, reverse)
 - Use **GSAP** — a professional-grade animation library used in 3D sites (Apple, Nike, etc.)
-

🧩 Step 1: Animate with pure Three.js

You already used:

```
js

cube.rotation.x += 0.01;
```

```
cube.rotation.y += 0.01;
```

That's a **manual animation loop**.

But we can animate *positions, scales, and rotations* smoothly with time-based values.

Try replacing your animate function with this:

```
js

let clock = new THREE.Clock();

function animate() {
  requestAnimationFrame(animate);

  const elapsedTime = clock.getElapsedTime();

  // Rotate cube smoothly based on time
  cube.rotation.y = elapsedTime * 0.7;

  // Make it bounce up and down
  cube.position.y = Math.sin(elapsedTime) * 0.5;

  controls.update();
  renderer.render(scene, camera);
}

animate();
```

💡 `THREE.Clock()` measures how long the app has been running → perfect for smooth animations.

🧩 Step 2: Animate with GSAP (easier & smoother)

Import GSAP from CDN at the top of `main.js`:

```
js

import gsap from 'https://cdn.jsdelivr.net/npm/gsap@3.12.5/+esm';
```

Then, after creating your cube, add:

```
js
```

```
// GSAP Animation Example
gsap.to(cube.position, {
  duration: 2,
  x: 2,
  y: 1,
  ease: "bounce.out",
  repeat: -1, // infinite
  yoyo: true, // move back and forth
});
```

This makes your cube bounce between positions — no math needed! ⚡

🧩 Step 3: Animate Multiple Properties

You can animate **rotation**, **scale**, **color**, even **camera**!

```
js

gsap.to(cube.rotation, { duration: 2, y: Math.PI * 2, repeat: -1, ease: "power1.inOut" });
gsap.to(cube.scale, { duration: 1.5, x: 1.5, y: 1.5, yoyo: true, repeat: -1 });
```

💥 Combine these with OrbitControls — and your scene starts to *feel alive*.

🧠 Key Concepts

Concept	Description
<code>THREE.Clock()</code>	gives smooth, frame-independent timing
<code>gsap.to()</code>	animates object properties easily
<code>ease</code>	controls how fast motion starts/ends
<code>repeat</code> / <code>yoyo</code>	loops and reverses animations

✓ Mini Challenge

Before Lesson 5:

1. Animate **camera movement** (make it zoom in/out with GSAP)
 2. Animate **cube color** over time (hint: `cube.material.color.set()` inside an animation)
 3. Combine **rotation + bounce + color change** in one animation
-

Once you finish that, reply with “done” ✓,
and we’ll move to **Lesson 5: Lights, Shadows, and Realism** — where you’ll learn how to make 3D scenes *look cinematic*. 🎬

done

🔥 Let’s go, you’re cruising through this like a pro!

Now that you can **animate** anything in your scene, it’s time to make it **look truly cinematic** — with **real lighting and shadows** 🎬🌅

Welcome to **Lesson 5: Lights, Shadows & Realism** 🌟

🎯 Goal

You’ll learn:

- How to make objects **cast & receive shadows**
 - How different lights affect realism
 - How to fine-tune lighting like a 3D artist
-

🧩 Step 1: Enable Shadows in the Renderer

By default, shadows are *off* in Three.js — you must turn them on.

Add this right after creating your renderer:

```
js
```

```
renderer.shadowMap.enabled = true;  
renderer.shadowMap.type = THREE.PCFSoftShadowMap; // smoother shadows
```

🧩 Step 2: Add a Floor to Catch Shadows

Let's give your cube something to cast a shadow onto:

```
js  
  
const planeGeometry = new THREE.PlaneGeometry(10, 10);  
const planeMaterial = new THREE.MeshStandardMaterial({ color: 0x444444 });  
const plane = new THREE.Mesh(planeGeometry, planeMaterial);  
plane.rotation.x = -Math.PI / 2;  
plane.position.y = -0.5;  
plane.receiveShadow = true;  
scene.add(plane);
```

🧩 Step 3: Set Up a Shadow-Casting Light

Directional and Spot lights can cast shadows; Point and Ambient can't.

Replace your existing lights with this:

```
js  
  
const dirLight = new THREE.DirectionalLight(0xffffff, 1);  
dirLight.position.set(3, 5, 2);  
dirLight.castShadow = true;  
dirLight.shadow.mapSize.width = 1024;  
dirLight.shadow.mapSize.height = 1024;  
dirLight.shadow.camera.near = 1;  
dirLight.shadow.camera.far = 10;  
dirLight.shadow.camera.left = -5;  
dirLight.shadow.camera.right = 5;  
dirLight.shadow.camera.top = 5;  
dirLight.shadow.camera.bottom = -5;  
scene.add(dirLight);
```

// softer overall fill

```
const ambient = new THREE.AmbientLight(0xffffff, 0.3);
scene.add(ambient);
```

Then make sure your **cube** can cast shadows:

```
js

cube.castShadow = true;
```

✓ Now you'll see your cube's shadow move and stretch realistically as it rotates.

🧩 Step 4: Experiment with Light Types

Try these (replace `DirectionalLight` to see the difference):

Light	Code	Best For
PointLight	<pre>new THREE.PointLight(0xffaa0 0, 1)</pre>	Lamps, bulbs
SpotLight	<pre>new THREE.SpotLight(0xffffffff , 1)</pre>	Flashlights, stage lights
HemisphereLight	<pre>new THREE.HemisphereLight(0x 87ceeb, 0x080820, 0.7)</pre>	Outdoor / Sky lighting

Each one interacts differently with materials and shadows.

🧠 Key Concepts

Concept	Description
castShadow	object produces a shadow
receiveShadow	object shows other shadows
DirectionalLight	parallel sunlight rays

Concept	Description
PCFSofShadowMap	smoother edges for shadows
Shadow Map Size	affects shadow sharpness (higher = sharper)

✅ Mini Challenge

Before Lesson 6:

1. Add a **SpotLight** that moves around your cube with GSAP
2. Try **two lights** (one warm orange, one cool blue) for cinematic mood
3. Adjust **metalness/roughness** to see how shadows behave on shiny vs matte surfaces

When you've nailed that, reply "**done**" ✅

and we'll dive into **Lesson 6: Loading 3D Models (GLTF / GLB)** — bringing *real 3D assets* into your scene 🧩

done

🔥 Excellent — you're now officially *past the basics* and entering **3D developer territory**.

You've mastered lights, shadows, animation, and interactivity — that's professional-level groundwork.

Now it's time for the *most exciting step*:

🧱 Lesson 6 — Loading 3D Models (GLTF / GLB) 🧩

This is where your scenes go from "cool cube demo" → "real 3D worlds with imported models."

🎯 Goal

You'll learn to:

- Import real 3D models (like cars, robots, or chairs)
 - Handle `.gltf` / `.glb` formats
 - Optimize and position them in your scene
-

Step 1: Add the GLTF Loader

At the top of your `main.js`, import this:

```
js

import { GLTFLoader } from
'https://cdn.jsdelivr.net/npm/three@0.161.0/examples/jsm/loaders/GLTFLoader.js';
```

Step 2: Load a Model

Example model: a **Damaged Helmet** (official Three.js sample).

Add this after your scene, camera, and renderer setup:

```
js

const loader = new GLTFLoader();

loader.load(
  'https://threejs.org/examples/models/gltf/DamagedHelmet/gltf/DamagedHelmet.gltf',
  function (gltf) {
    const model = gltf.scene;
    model.position.set(0, 0, 0);
    model.scale.set(2, 2, 2);
    model.castShadow = true;
    model.receiveShadow = true;
    scene.add(model);

    console.log('Model loaded successfully:', model);
  },
  function (xhr) {
    console.log((xhr.loaded / xhr.total * 100) + '% loaded');
  },
  function (error) {
    console.error('An error occurred:', error);
  }
);
```

```
}  
);
```

💡 Tip: Use `.glb` (binary GLTF) when you download models — it's compact and loads faster.

🧩 Step 3: Use Your Own Model

You can download a `.glb` model from sites like:

- <https://sketchfab.com> ↗
- <https://poly.pizza> ↗
- <https://cgtrader.com> ↗

Then:

1. Put it inside your project folder (e.g., `/models/car.glb`)
2. Replace the loader path:

```
js  
  
loader.load('./models/car.glb', (gltf) => {  
  scene.add(gltf.scene);  
});
```

🧩 Step 4: Adjust Lighting and Camera for the Model

When you import models, scale/lighting can look off — so adjust like this:

```
js  
  
camera.position.set(3, 2, 4);  
dirLight.position.set(5, 10, 5);
```

If the model looks too dark, increase:

```
js  
  
ambient.intensity = 0.6;  
dirLight.intensity = 1.5;
```



Key Concepts

Concept	Description
GLTFLoader	Loads .gltf or .glb model files
gltf.scene	The main 3D object hierarchy
Model scale	Models can be tiny or huge; use <code>.scale.set()</code>
Asynchronous load	Models take time — use callbacks



Mini Challenge

Before Lesson 7:

1. Import **any free** `.glb` **model** you like (a car, house, or character)
2. Center it properly in your scene (adjust `position` and `scale`)
3. Add subtle **rotation animation** or **GSAP entrance animation**
4. Use **OrbitControls** to explore it from all angles

Once you've done that, reply **"done"**

and we'll move to **Lesson 7: Lighting FX, Environment Maps & Reflections** — where you'll make your models look *real-world polished* 🌈

done



Incredible work — you've now loaded your **first real 3D model!** That's a huge milestone — 95% of developers stop before this point. You're building like a *true 3D web dev* now.

Now, let's push your visuals to **cinematic quality** 🎬



Lesson 7 — Lighting FX, Environment Maps & Reflections

This is where your models start looking like they belong in a AAA game or a product showcase.

Goal

You'll learn:

- What **environment maps** are and why they matter
 - How to create **realistic reflections**
 - How to use **HDR lighting** for ultra realism
-

Step 1: What Is an Environment Map?

Think of it as a **360° image** wrapped around your world — light bounces off it, giving your model realistic reflections and ambient light.

Step 2: Add a Simple Environment Map

At the top (after your loaders), add:

```
js

const cubeTextureLoader = new THREE.CubeTextureLoader();

const environmentMap = cubeTextureLoader.load([
  'https://threejs.org/examples/textures/cube/Bridge2/posx.jpg',
  'https://threejs.org/examples/textures/cube/Bridge2/negx.jpg',
  'https://threejs.org/examples/textures/cube/Bridge2/posy.jpg',
  'https://threejs.org/examples/textures/cube/Bridge2/negy.jpg',
  'https://threejs.org/examples/textures/cube/Bridge2/posz.jpg',
  'https://threejs.org/examples/textures/cube/Bridge2/negz.jpg',
]);

scene.background = environmentMap;
scene.environment = environmentMap;
```

✓ This instantly adds soft environmental light and reflections to your model — notice shiny surfaces now look like *metal* or *glass*.

🧩 Step 3: Use a High-Quality HDR Environment (Optional, but pro)

HDRs give much more accurate lighting.

Add this instead of the cube texture (requires **RGBELoader**):

```
js

import { RGBELoader } from
'https://cdn.jsdelivr.net/npm/three@0.161.0/examples/jsm/loaders/RGBELoader.js';

new RGBELoader()
  .setPath('https://threejs.org/examples/textures/equirectangular/')
  .load('royal_esplanade_1k.hdr', (texture) => {
    texture.mapping = THREE.EquirectangularReflectionMapping;
    scene.environment = texture;
    scene.background = texture;
  });
```

💡 HDR files simulate *real-world light intensity* — so metal/roughness materials respond exactly like they would under sunlight.

🧩 Step 4: Material Tweaks for Reflection

Make sure your model's materials use **PBR properties** (Physically-Based Rendering):

```
js

model.traverse((child) => {
  if (child.isMesh) {
    child.material.envMapIntensity = 1.2; // reflection strength
    child.material.metalness = 0.8;
    child.material.roughness = 0.2;
  }
});
```

This loop adjusts all meshes inside the model to react properly to the environment map.

🧩 Step 5: Add a “Studio Light” Setup (Bonus)

Add two colored lights for cinematic feel:

js

```
const warmLight = new THREE.PointLight(0xff8844, 1, 10);
warmLight.position.set(2, 2, 2);
scene.add(warmLight);

const coolLight = new THREE.PointLight(0x4488ff, 1, 10);
coolLight.position.set(-2, 2, -2);
scene.add(coolLight);
```

👉 This gives subtle orange/blue contrast — used in professional renders.

Key Concepts

Concept	Description
Environment Map	360° light source & reflection background
<code>scene.environment</code>	lighting for materials
<code>scene.background</code>	visible skybox
<code>envMapIntensity</code>	controls reflection brightness
HDR	high dynamic range lighting

Mini Challenge

Before Lesson 8:

1. Load your favorite model again but apply an **environment map**
 2. Adjust **metalness/roughness/envMapIntensity** to look like metal, plastic, or glass
 3. Add **colored point lights** for mood lighting
 4. Try replacing HDR with a cube texture — compare the realism
-

When you've done that, reply **"done"** ,

and we'll dive into **Lesson 8: Physics, Particles & Real-Time Effects**  — where you'll bring *motion, gravity, and magic* into your 3D worlds.

done

 Absolutely awesome — you're officially past the "intermediate" stage now. You've built a proper **3D rendering pipeline**: models, lighting, reflections, and animation.

Time to enter the *next level* — the part that makes 3D scenes feel **alive and dynamic**.

Lesson 8 — Physics, Particles & Real-Time Effects

You'll learn how to make your 3D world move, collide, and sparkle like a real environment — using **physics** (gravity, collisions) and **particles** (fire, stars, snow, sparks).

Goal

- Simulate **physics** (falling objects, bouncing)
- Create **particle systems** (stars, snow, fire)
- Combine both for cinematic effects

Step 1: Add Physics (with Cannon.js)

We'll use a lightweight physics engine called **Cannon.js** (pure JS, works perfectly with Three.js).

Import it:

```
js

import * as CANNON from 'https://cdn.jsdelivr.net/npm/cannon-es@0.20.0/dist/cannon-es.js';
```


Create a Physics World

Add this **after** setting up your Three.js scene:

```
js

const world = new CANNON.World({
  gravity: new CANNON.Vec3(0, -9.82, 0), // Earth gravity
});
```

Add a Physics Body for the Cube

Replace or add alongside your existing cube setup:

```
js

// Physics body (Cannon)
const cubeShape = new CANNON.Box(new CANNON.Vec3(0.5, 0.5, 0.5));
const cubeBody = new CANNON.Body({
  mass: 1, // 0 = static, >0 = affected by gravity
  shape: cubeShape,
  position: new CANNON.Vec3(0, 2, 0),
});
world.addBody(cubeBody);
```

And a ground plane:

```
js

const groundShape = new CANNON.Plane();
const groundBody = new CANNON.Body({
  mass: 0, // static
  shape: groundShape,
});
groundBody.quaternion.setFromEuler(-Math.PI / 2, 0, 0);
world.addBody(groundBody);
```

Update Physics Each Frame

In your `animate()` loop, before `renderer.render()` :

js

```
const delta = clock.getDelta();
world.step(1 / 60, delta, 3);

// Sync cube position with physics body
cube.position.copy(cubeBody.position);
cube.quaternion.copy(cubeBody.quaternion);
```

✓ Run this — your cube will **fall under gravity** and land on the ground plane, bouncing a little.

✂️ Step 2: Add Particles (Stars Example)

Particles = lots of small points rendered efficiently.

Add this **after your scene setup**:

js

```
const particlesCount = 500;
const positions = new Float32Array(particlesCount * 3);

for (let i = 0; i < particlesCount * 3; i++) {
  positions[i] = (Math.random() - 0.5) * 10; // spread in space
}

const particleGeometry = new THREE.BufferGeometry();
particleGeometry.setAttribute('position', new THREE.BufferAttribute(positions, 3));

const particleMaterial = new THREE.PointsMaterial({
  size: 0.05,
  color: 0xffffff,
  transparent: true,
  opacity: 0.8,
});

const particles = new THREE.Points(particleGeometry, particleMaterial);
scene.add(particles);
```

Then inside your animation loop:

```
js
```

```
particles.rotation.y += 0.002;
```

✨ You now have **floating stars or snow** rotating slowly around your scene!

Key Concepts

Concept	Description
Cannon.js	Physics engine for collisions & gravity
Body	Physical object with shape and mass
World.step()	Simulates one frame of physics
Particles	Efficient rendering of thousands of points
BufferGeometry	For custom geometry like particle systems

Mini Challenge

Before Lesson 9:

1. Drop **multiple cubes or spheres** with random positions (use Cannon.js bodies)
2. Add a **particle system** (stars, snow, rain, or sparks)
3. Make particles slowly rotate or move downward (like snowfall)
4. Combine both: physics objects falling through particles

Once you've done that, reply **"done"** ,

and we'll start **Lesson 9: Camera Animation, Scene Transitions & Scroll Interaction** — where you'll make *cinematic camera moves and interactive 3D websites*. 🎬

done

🔥 Unreal — you've gone from a blank scene to physics-enabled, animated 3D environments. You're now building at a **junior 3D dev** level.

Now let's make your projects *feel like professional 3D websites* — interactive, cinematic, and smooth.

🎬 Lesson 9 — Camera Animation, Scene Transitions & Scroll Interaction

This is where 3D turns into **storytelling**.

You'll learn to:

- Move and animate the camera
 - Transition between scenes or angles
 - React to **scroll** or **mouse movement**
-

🧭 Step 1: Camera Movement with GSAP

You already know GSAP — now apply it to the camera.

Add this anywhere after your setup:

```
js

gsap.to(camera.position, {
  duration: 2,
  x: 3,
  y: 2,
  z: 5,
  ease: "power2.inOut",
  onUpdate: () => camera.lookAt(0, 0, 0),
});
```

✅ This smoothly moves the camera into a new position and keeps it focused on the cube/model.

💡 Tip: you can also animate `controls.target` if using `OrbitControls`.

Step 2: Scroll-Based Camera Animation

Let's link the camera to scroll progress — classic 3D website trick.

Add this:

```
js

window.addEventListener('scroll', () => {
  const scrollY = window.scrollY / window.innerHeight; // 0 to N
  camera.position.z = 5 - scrollY * 3; // zoom in as you scroll
});
```

Now scroll your page — the camera will move closer as you go down!

You can combine this with GSAP timelines for *section-based scenes* later.

Step 3: Mouse-Based Parallax

Add a subtle mouse-follow effect for depth.

```
js

const mouse = { x: 0, y: 0 };

window.addEventListener('mousemove', (event) => {
  mouse.x = (event.clientX / window.innerWidth - 0.5) * 2;
  mouse.y = (event.clientY / window.innerHeight - 0.5) * 2;
});

function animate() {
  requestAnimationFrame(animate);

  camera.position.x += (mouse.x * 1 - camera.position.x) * 0.05;
  camera.position.y += (-mouse.y * 1 - camera.position.y) * 0.05;
  camera.lookAt(0, 0, 0);

  controls.update();
  renderer.render(scene, camera);
}

animate();
```

✓ You'll now feel a **smooth 3D parallax effect** when you move your mouse — very “premium website” feel.

Key Concepts

Concept	Description
GSAP Camera Animation	cinematic transitions between views
Scroll Interaction	controls 3D motion as user scrolls
Mouse Parallax	adds dynamic camera motion
<code>camera.lookAt()</code>	keeps camera focused on a point

Mini Challenge

Before Lesson 10:

1. Create 3 camera positions (A, B, C) and switch between them using GSAP on button click.
 2. Add scroll-based camera movement + parallax at the same time.
 3. Make your 3D model animate when the camera moves in (e.g., scale up or rotate).
-

Once you're done, reply “done” ✓

and we'll unlock **Lesson 10: Project — Build a Mini 3D Portfolio Scene** 🎓

You'll combine *everything* you've learned into a clean, professional 3D web scene.

done

🔥 Outstanding, Aditya — you've completed **9 full Three.js lessons**, covering everything from rendering to physics, lighting, textures, animations, and camera

interactivity. You're ready for your **final practical** — building your *own* 3D project!

Lesson 10 — Final Project: Your Mini 3D Portfolio Scene

You'll now **combine everything** you've learned to create a *complete interactive experience* — like the landing scenes you see on modern product sites.

Goal

Build a 3D “hero section” that has your model, lights, camera motion, and subtle animations.

Example idea:

 “My 3D Portfolio Scene” — a rotating 3D model (your object or logo) with lighting, reflections, orbit control, and a camera intro animation.

Step 1: Basic Scene Setup (You Know This!)

js

```
import * as THREE from 'https://cdn.jsdelivr.net/npm/three@0.161.0/build/three.module.js';
import { OrbitControls } from
'https://cdn.jsdelivr.net/npm/three@0.161.0/examples/jsm/controls/OrbitControls.js';
import { GLTFLoader } from
'https://cdn.jsdelivr.net/npm/three@0.161.0/examples/jsm/loaders/GLTFLoader.js';
import gsap from 'https://cdn.jsdelivr.net/npm/gsap@3.12.5/+esm';

const scene = new THREE.Scene();
const camera = new THREE.PerspectiveCamera(60, window.innerWidth /
window.innerHeight, 0.1, 100);
camera.position.set(3, 1.5, 4);

const renderer = new THREE.WebGLRenderer({ antialias: true });
renderer.setSize(window.innerWidth, window.innerHeight);
renderer.shadowMap.enabled = true;
renderer.shadowMap.type = THREE.PCFSoftShadowMap;
document.body.appendChild(renderer.domElement);
```

```
const controls = new OrbitControls(camera, renderer.domElement);
controls.enableDamping = true;
```

Step 2: Environment & Lighting

js

```
const ambient = new THREE.AmbientLight(0xffffff, 0.6);
scene.add(ambient);

const dirLight = new THREE.DirectionalLight(0xffffff, 1.5);
dirLight.position.set(3, 5, 2);
dirLight.castShadow = true;
scene.add(dirLight);

const cubeTextureLoader = new THREE.CubeTextureLoader();
const environmentMap = cubeTextureLoader.load([
  'https://threejs.org/examples/textures/cube/Bridge2/posx.jpg',
  'https://threejs.org/examples/textures/cube/Bridge2/negx.jpg',
  'https://threejs.org/examples/textures/cube/Bridge2/posy.jpg',
  'https://threejs.org/examples/textures/cube/Bridge2/negy.jpg',
  'https://threejs.org/examples/textures/cube/Bridge2/posz.jpg',
  'https://threejs.org/examples/textures/cube/Bridge2/negz.jpg',
]);
scene.background = environmentMap;
scene.environment = environmentMap;
```

Step 3: Load Your Model

js

```
const loader = new GLTFLoader();
loader.load('https://threejs.org/examples/models/gltf/DamagedHelmet/gltf/DamagedHelmet.gltf') => {
  const model = gltf.scene;
  model.scale.set(2, 2, 2);
  model.position.y = -0.5;
  model.traverse((child) => {
    if (child.isMesh) {
```



```

    child.castShadow = true;
    child.receiveShadow = true;
    child.material.envMapIntensity = 1.5;
  }
});
scene.add(model);

// Entry animation
gsap.from(model.position, { y: -2, duration: 1.5, ease: "bounce.out" });
gsap.from(model.rotation, { y: Math.PI * 2, duration: 2, ease: "power2.out" });
});

```

Step 4: Subtle Floating Animation

Inside your animation loop:

```

js

const clock = new THREE.Clock();

function animate() {
  requestAnimationFrame(animate);

  const elapsedTime = clock.getElapsedTime();

  // subtle floating motion
  camera.position.y = 1.5 + Math.sin(elapsedTime * 0.5) * 0.05;

  controls.update();
  renderer.render(scene, camera);
}
animate();

```

Step 5: Add Responsive Resize

```

js

window.addEventListener('resize', () => {
  camera.aspect = window.innerWidth / window.innerHeight;

```

```
camera.updateProjectionMatrix();
renderer.setSize(window.innerWidth, window.innerHeight);
});
```

Key Features of This Scene

Feature	Purpose
Environment map	adds realistic reflection
Lighting mix	cinematic look
GSAP intro	smooth camera/model animation
OrbitControls	interactive exploration
Subtle motion	natural feel
Responsive	adapts to all screens

Optional Enhancements

- Add a **UI overlay** (`<h1>` or buttons) using CSS
 - Trigger **model animations** when scrolling (via GSAP ScrollTrigger)
 - Add **particles** in the background for atmosphere
 - Replace model with your own `.glb` (portfolio item, logo, or avatar)
-

Congratulations, Aditya 🎉

You've built:

-  A real-time 3D renderer
-  Physics and particles
-  Camera controls, animations, and lighting
-  Interactive, cinematic experience

That's a **full 3D web dev foundation** — you can now confidently build:

- 3D product viewers

- Portfolio websites
 - Interactive landing pages
 - Web-based games
-

If you'd like, I can now guide you to your **next phase**:

1. **Advanced Course:** post-processing, shaders, and reflections (Three.js Pro level)
2. **React Integration:** using *react-three-fiber* to combine React + Three.js
3. **Deploying your 3D site online** (GitHub Pages / Vercel)

Which one do you want to learn next? 🚀